

镜像仓库 Registry

第 6 讲

目标

让镜像跨机器、跨团队地流动起来

- 厘清:Registry(仓库) vs Repository(仓储) 的区别
- 读懂镜像**全名**: 三部分分段 + 默认值
- Docker Hub 闭环: build → tag → login → push → pull → run
- 用 `registry` 镜像**一行起飞**本地私有仓库

一、什么是镜像仓库

存放并分发镜像的"图书馆", 命名靠全名

Registry 是"店", Repository 是"货架上的一个 App"

概念	中文	作用	类比
Registry	镜像仓库/注册中心	存放分发镜像的 服务器	应用商店
Repository	仓储/命名空间项目	仓库里按名组织的一个镜像集合	商店里的一个 App

Registry(一台服务器)

- repository: nginx → 多个 tag: 1.25 / 1.26 / latest
- repository: alpine
- repository: 你的镜像名



镜像全名, 其实是三段式地址

[仓库地址]/[命名空间]/[镜像名]:[标签]

registry / owner / repo : tag

docker.io/library/python:3.12-slim



部分	可省略?	默认值
仓库地址	是	docker.io (Docker Hub)
命名空间	是	library (官方空间)
标签	是	latest

```
docker pull python:3.12-slim ≡ docker pull docker.io/library/python:3.12-slim
```

为什么需要仓库

需求	仓库如何满足
分发	不同机器/团队从同一处拉同一份镜像
备份	镜像集中托管, 不易丢
协作	推送即共享, 免去线下拷文件
版本管理	用 tag 管版本、namespace 分归属
私有化	企业自建, 镜像不出内网

公共仓库 vs 私有仓库

维度	Docker Hub(公共)	私有仓库
访问	公开, 任人拉取	需认证 / 可信网络
成本	公共免费, 私有钱配额	自己掌控, 无配额
安全	有被污染风险	完全自控, 适内网
实现	Docker Hub	Harbor · Nexus · 阿里云 ACR · 本讲 <code>registry</code>

二、Docker Hub 与登录

默认公共仓库, 推需登录, 拉公共镜像不用

login 登录 · search 搜镜像

```
docker login          # 交互式输入 用户名/密码
docker logout         # 退出登录

docker search nginx   # 搜官方镜像
docker search --filter=stars=100 alpine # 按 Star 过滤
```



- 登录凭证存 `~/.docker/config.json`
- 推私有仓库要 login; 拉公共镜像不需要

推送的身份红线：命名空间必须是你自己

```
docker push myname/myimg:1.0 # ✅ 命名空间 = 你的账号  
docker push ubuntu:1.0      # ❌ 无权推 library/ubuntu
```



镜像名里的命名空间 = 你, 才推得进; 否则被拒

三、Docker Hub

推送/拉取完整流程

build → tag → login → push → 异地 pull → run

一波操作串起完整闭环

```
docker login
docker build -t flask-demo:1.0 .
docker tag flask-demo:1.0 jamelouis/flask-demo:1.0
docker push jamelouis/flask-demo:1.0

# 另一台机器的世界
docker pull jamelouis/flask-demo:1.0
docker run -d -p 5000:5000 jamelouis/flask-demo:1.0
```

① 登录
② 构建
③ 加用户名
④ 推送

⑤ 拉取
⑥ 运行



tag 只加别名不复制数据; 推送按层上传, 共享层可跳过

四、本地私有仓库

registry 镜像一行起飞, 镜像不出内网

一行命令起一个私有仓库

```
docker pull registry:2
docker run -d --name myregistry -p 5000:5000 registry:2

docker ps                                # 仓库容器在跑
curl http://localhost:5000/v2/          # 返回 {} 服务正常
```



适合: 教学实验 · 内网/离线 · 含私有代码 · 快速迭代

推到本地：前缀一变, 目标即换

```
docker tag flask-demo:1.0 localhost:5000/flask-demo:1.0
docker push localhost:5000/flask-demo:1.0      # 打到本地!

docker rmi localhost:5000/flask-demo:1.0
docker pull localhost:5000/flask-demo:1.0      # 从本地拉回
```



全名里"仓库地址"字段的意义在此: 前缀 localhost:5000/ 即本地

数据持久化: 挂一卷, 删容器数据不丢

```
docker run -d --name myregistry -p 5000:5000 \  
-v /opt/registry:/var/lib/registry \  
registry:2
```



- `/var/lib/registry` 是 registry 内部存镜像数据的目录
- 挂到宿主 `/opt/registry`, 删容器再启数据仍在

局域网其他主机访问, 需在客户端配 `insecure-registries` (了解即可)

五、仓库命令速查

一张表记牢 push/pull 的动静

命令一览

命令	作用
<code>login / logout</code>	登录 / 退出仓库
<code>pull <全名></code>	从仓库拉取(默认 Docker Hub)
<code>push <全名></code>	推送到仓库
<code>search <关键词></code>	在仓库搜索
<code>tag <原名> <新名></code>	加别名/命名空间
<code>rmi</code>	删除本地镜像
<code>curl host:5000/v2/_catalog</code>	列本地仓库镜像列表

六、实战一

推一个镜像到 Docker Hub

走通"构建 → 推送 → 异地拉取"真正的完整闭环

打上自己的命名空间, 推送, 拉回验证

```
cd demo
docker build -t flask-demo:1.0 .      # ① 构建(见 Dockerfile 讲)
docker login                          # ② 登录

docker tag flask-demo:1.0 <你的用户名>/flask-demo:1.0  # ③ 加用户名
docker images                        # ④ 两条同 ID 别名共存

docker push <你的用户名>/flask-demo:1.0  # ⑤ 推送

docker rmi <你的用户名>/flask-demo:1.0  # ⑥ 删本地, 模拟没有它
docker pull <你的用户名>/flask-demo:1.0  # ⑦ 从 Hub 拉回
docker run -d -p 5000:5000 <你的用户名>/flask-demo:1.0
```



七、实战二：搭建并体验本地私有仓库

不碰公网, 亲手推拉, 彻骨理解全名与地址

起仓库 → 推 alpine → 删本地 → 从仓库拉回

```
docker run -d --name myregistry -p 5000:5000 \
  -v /opt/registry:/var/lib/registry registry:2 # ① 起仓库
curl http://localhost:5000/v2/_catalog # ② 空列表 []

docker tag alpine:3.20 localhost:5000/alpine:3.20 # ③ 加本地前缀
docker push localhost:5000/alpine:3.20 # ④ 推进仓库
curl http://localhost:5000/v2/_catalog # ⑤ 见 "alpine"

docker rmi localhost:5000/alpine:3.20 alpine:3.20 # ⑥ 删本地两镜像
docker pull localhost:5000/alpine:3.20 # ⑦ 从本地拉回
docker run --rm localhost:5000/alpine:3.20 echo "hello"
```



数据靠卷幸存: 删仓库容器, alpine 还在

```
docker rm -f myregistry          # ① 删仓库容器
curl -s http://localhost:5000/v2/_catalog | head  # ② 服务停, 访问失败

docker run -d --name myregistry -p 5000:5000 \
  -v /opt/registry:/var/lib/registry registry:2 # ③ 同一卷重建
curl http://localhost:5000/v2/_catalog          # ④ alpine 还在!
```



一行束清可选清场: `docker rm -f myregistry ; sudo rm -rf /opt/registry`

一张表带走全部要点

方面	要点
仓库	Registry=服务器的"店", Repository=里面一个"App", 别混
镜像全名	[地址]/[空间]/[名字]:[标签], 省略默认 docker.io/library/latest
Docker Hub	推需 login + 命名空间=自己; 拉公共镜像不用
完整流程	build → tag → login → push → (异地)pull → run
本地仓库	registry:2 一行起飞; 前缀改 localhost:5000/ 即推本地; 挂 /var/lib/registry 持久化

图纸有了, 房子会盖了, 图纸也流通了

下讲: 卷(Volume)、网络(Network)与 Compose 编排——让多容器协作成一套应用

谢谢大家

Q & A